

Buổi 5

DSU và Fenwick tree, bản Python

Kiến thức, mười bài tập và mười code mẫu

Ngày 23 tháng 9 năm 2026

TS. Đỗ Phúc Hảo

Tài liệu một file cho buổi DSU và Fenwick tree, bản Python. Gồm bốn phần: kiến thức, mười đề bài, gợi ý và lời giải, và mười code mẫu đầy đủ.

Đề bài và bộ test dùng chung với bản C++ ở `../topics/05-dsu-fenwick/`. Cùng một đề, cùng một bộ test, khác ngôn ngữ. Hai cấu trúc của buổi này có chung một điểm quý: code rất ngắn, chưa tới hai chục dòng mỗi cái, nhưng mở ra một lớp bài rất rộng.

Điểm khác lớn nhất so với bản C++ nằm ở **một hàm duy nhất**: hàm `find` của DSU. Bản C++ viết nó thế nào cũng được. Bản Python thì không, và mục 3 là chỗ nói chuyện đó bằng số đo chứ không bằng lời khuyên.

1. Mục lục

Phần	Nội dung
I	Kiến thức, mười ba mục
II	Mười đề bài A tới J
III	Gợi ý phân tầng và lời giải từng bài
IV	Mười code mẫu đầy đủ

Phần I. Kiến thức

1. Hai cấu trúc, và lớp bài chúng mở ra

DSU, hay cấu trúc các tập rời nhau, quản lý một họ các nhóm với hai thao tác: gộp hai nhóm, và hỏi hai phần tử có cùng nhóm không. Nó trả lời cả hai trong thời gian gần như hằng số.

Fenwick tree, còn gọi là BIT, đặt lên một dãy số và cho phép vừa sửa một phần tử vừa hỏi tổng một đoạn, mỗi thao tác $O(\log n)$.

Điểm chung của hai cấu trúc: chúng thay một vòng lặp $O(n)$ bên trong một vòng lặp q bước bằng một quãng đường ngắn, và **điều đó quan trọng với Python hơn với C++**. Một thuật toán $O(n \cdot q)$ với $n = q = 2 \cdot 10^5$ là $4 \cdot 10^{10}$ phép tính; C++ cũng chết, còn Python thì chết theo một cách không ai chờ nổi.

Điểm khác: Fenwick tính truy vấn đoạn bằng **hiệu hai tổng tiền tố**, nên phép toán bắt buộc phải có nghịch đảo. Cộng thì có, lấy giá trị lớn nhất thì không. Đó là lý do bài cực trị đoạn phải chờ tới buổi segment tree.

2. DSU, ý tưởng và hai tối ưu

Mỗi nhóm là một cây, mỗi phần tử trở tới cha, gốc cây đại diện cho cả nhóm. Hai phần tử cùng nhóm khi và chỉ khi chúng có cùng gốc.

Toàn bộ cấu trúc, bản Python, là hai list và hai đoạn vòng lặp:

```
p = list(range(n + 1))    # p[v] is the parent of v, a root points at itself
sz = [1] * (n + 1)        # sz[v] is the size of the tree, valid ONLY at a root

# find the root of a, compressing the path on the way
while p[a] != a:
    p[a] = p[p[a]]         # point a at its grandparent
    a = p[a]

# unite the roots a and b
if a != b:
    if sz[a] < sz[b]:
        a, b = b, a        # hang the smaller tree under the bigger one
    p[b] = a
    sz[a] += sz[b]
```

Hai tối ưu, và cả hai đều cần:

Nén đường. Trên đường đi tìm gốc, cho mỗi nút trở thẳng lên ông nó. Dạng viết trên gọi là **chia đôi đường**: nó chỉ đi một lượt, vừa đi vừa rút ngắn. Lần sau đi lại chuỗi ấy sẽ ngắn hơn hẳn.

Hợp theo kích thước. Luôn treo cây nhỏ dưới cây lớn. Làm ngược thì cây có thể dài thành một chuỗi và mỗi lần tìm gốc tốn $O(n)$.

Biết một phép hợp có thật sự gộp hai nhóm khác nhau hay không là thông tin quý nhất của cấu trúc này. Bản C++ gói nó thành giá trị trả về của unite; bản Python ở đây không gói vào hàm, nên thông tin ấy chính là điều kiện `a != b` ngay sau hai vòng tìm gốc. Riêng nó giải xong bài B, bài D và bài E.

3. Vì sao find viết LẶP, và con số đáng nhớ không phải con số tốc độ

Sách vở viết find bằng đệ quy, ba dòng, đẹp và dễ nhớ:

```
def find(v):
    if p[v] == v:
        return v
    p[v] = find(p[v])    # DO NOT ship this in Python
    return p[v]
```

Ở Python có hai lý do để không dùng nó, và chỉ **một** trong hai là thật.

Lý do hay bị nêu, và nó gần như không đúng: tốc độ. Đo trên test 09 của bài A, tức $n = q = 2 \cdot 10^5$, lấy giá trị tốt nhất của năm lượt chạy:

Cách viết find	Thời gian
đệ quy, để trong một hàm	0,09 s
lặp hai lượt, để trong một hàm	0,10 s
lặp chia đôi đường, gõ thẳng vào main	0,08 s

Chênh lệch là hai phần trăm giây trên tổng số 0,09 giây, và phần lớn khoản đó là chi phí **gọi hàm** chứ không phải chi phí đệ quy. Lý do thì do được luôn: trên chính test ấy, quãng đường dài nhất mà một lần find phải đi là **5 nút**, trung bình **0,283 nút** trên 400.000 lần gọi. Hợp theo kích thước cộng nén đường làm cây phẳng tới mức hình dạng của find gần như không còn ý nghĩa.

Tách riêng phần find ra khỏi mọi thứ khác thì mới thấy được chênh lệch. Dựng 200.000 nút thành các chuỗi dài D, rồi gọi find một lần từ đáy mỗi chuỗi, tức tổng số bước là như nhau ở mọi dòng của bảng:

Độ sâu D	đệ quy	lặp hai lượt	lặp chia đôi
1	0,013 s	0,012 s	0,011 s
8	0,011 s	0,019 s	0,008 s
64	0,012 s	0,020 s	0,008 s
1.000	0,015 s	0,020 s	0,008 s
20.000	0,018 s	0,018 s	0,007 s

Đọc bảng này cho ra một kết quả **ngược** với câu thường nghe. Đệ quy không hề chậm hơn vòng lặp; cái chậm là **lặp hai lượt**, vì nó đi hết đường một lần để tìm gốc rồi đi lại lần nữa để nén. Dạng chia đôi đường nhanh nhất ở mọi độ sâu, và nó nhanh nhất vì nó đi **một** lượt. Nếu bạn định bỏ đệ quy chỉ để lấy tốc độ thì bạn đã bỏ nhầm thứ.

Lý do thật: ngăn xếp. Giới hạn đệ quy mặc định của Python là 1.000 mức. Nâng bằng sys.setrecursionlimit không giải quyết gì, nó chỉ dời chỗ chết: CPython khi ấy tràn **ngăn xếp thật của hệ điều hành**. Do trên máy dựng tài liệu này, bằng cách chia đôi khoảng:

Độ sâu chuỗi	Kết quả
32.481	chạy xong bình thường
32.870	chết, tín hiệu 11, không một dòng truy vết

Không có RecursionError, không có thông báo, không có gì trên màn hình. Tiến trình biến mất với mã thoát 139. Bộ chấm chỉ thấy một mã thoát khác 0 nên nó báo **RE**, và người bị chấm không có manh mối nào để lần.

Và đây là chỗ nguy hiểm thật sự: một dữ liệu hợp lệ của chính bài A ép được chuyện đó. Lấy $n = q = 2 \cdot 10^5$ rồi cho q - 1 lệnh gộp dạng 1 i i-1 với i chạy từ 2 tới n, kết lại bằng một truy vấn 2 1 n. Nếu người viết bỏ hợp theo kích thước thì mỗi lệnh gộp treo cả cây cũ xuống dưới một nút mới, mỗi lệnh chỉ tốn 0(1) nên không có gì chậm, nhưng cây dài dần thành một chuỗi 200.000 nút. Tới truy vấn cuối:

Chương trình	Kết quả trên dữ liệu ấy
find đệ quy, không hợp theo kích thước	mã thoát 139, không in gì, 0,13 s
find chia đôi đường, không hợp theo kích thước	in YES, 0,07 s
bản mẫu của buổi này, có cả hai tối ưu	in YES, 0,08 s

Chuỗi ấy tồn tại được chính vì nén đường chưa kịp chạm vào nó: các lệnh gộp chỉ đụng tới gốc và tới một nút lẻ, còn lần đầu tiên có ai đi hết chuỗi thì cũng là lần chương trình chết.

Kết luận dùng được. Viết find bằng vòng lặp không phải vì nó nhanh hơn, mà vì nó **không chết**. Con số đáng nhớ của mục này là **32.000**, không phải một tỉ lệ tốc độ. Và con số đó nhỏ hơn n của gần như mọi đề trong kho.

4. Vì sao DSU gần như $O(1)$

Với cả hai tối ưu, chi phí khấu hao mỗi thao tác là $O(a(n))$, trong đó a là hàm ngược Ackermann. Với mọi n mà máy tính chứa nổi, $a(n)$ không vượt quá 4.

Bạn không cần chứng minh được điều đó, chỉ cần hai hệ quả thực hành:

- DSU coi như miễn phí, cứ gọi thoải mái trong vòng lặp.
- Bỏ một trong hai tối ưu thì tụt xuống $O(\log n)$, vẫn ổn về thời gian; **nhưng nếu find của bạn là đệ quy thì $O(\log n)$ đó lại chính là thứ giết chương trình**, vì $\log$ ở đây là độ sâu ngăn xếp. Bỏ cả hai thì thành $O(n)$, và với Python đó là RE chứ không phải TLE.

Hai dòng trên là chỗ bản Python khác bản C++ nhiều nhất. Ở C++, bỏ tối ưu là chuyện thời gian. Ở Python, nó là chuyện sống chết.

5. Bốn bài toán DSU điển hình

Bốn bài đầu của buổi khác nhau đúng một dòng, và đó là lý do nên làm chúng liên nhau.

Bài	Câu hỏi	Một dòng khác nhau
A	hai đỉnh có nối nhau không	so hai gốc sau khi tìm
B	số thành phần liên thông	comp -= 1 khi a != b
C	kích thước nhóm lớn nhất	so best với sz[a] sau khi gộp
D	số cạnh thừa	thua += 1 khi a == b

Ba điều chung cho cả bốn:

- **Không cần duyệt gì ở cuối.** Số thành phần bắt đầu bằng n và giảm dần; kích thước lớn nhất chỉ tăng nên không bao giờ phải tính lại.
- **sz chỉ đúng tại gốc.** Ở một nút không phải gốc, nó giữ giá trị cũ từ lúc nút ấy còn là gốc. Trong code của kho này, biến a sau hai vòng while đã là gốc, nên $sz[a]$ hợp lệ; nếu bạn tự viết hàm find thì phải nhớ gọi nó trước khi đọc sz.
- **Một phép gộp chỉ đổi đúng một nhóm**, nên một phép so sánh mỗi cạnh là đủ.

6. Kruskal, cây khung nhỏ nhất

Bài E. Sắp cạnh theo trọng số tăng dần, duyệt và lấy cạnh nào nối hai thành phần khác nhau.

Đây chính là tham lam của buổi trước, và đối số đối chỗ áp thẳng vào: **cạnh nhẹ nhất bắc qua một lát cắt bất kỳ luôn thuộc một cây khung nhỏ nhất nào đó**. Nếu một cây khung tối ưu không chứa cạnh đó, thêm nó vào sẽ tạo một chu trình đi qua lát cắt, và bỏ cạnh nặng hơn trên chu trình ấy cho ra cây khung không nặng hơn.

Điều kiện "đã dùng đúng $n - 1$ cạnh" là cách kiểm đồ thị có liên thông hay không, miễn phí.

Phần Python của bài này nằm ở chỗ sắp xếp. Cả m cạnh phải thành một list tuple rồi sắp. Viết vòng lặp Python để dựng list ấy là làm từng đối tượng một; thay bằng một lời gọi duy nhất:

```
canh = data[2:2 + 3 * m]
e = sorted(zip(map(int, canh[2::3]), map(int, canh[0::3]), map(int, canh[1::3])))
```

Trọng số đặt ở vị trí **đầu** tuple, nhờ vậy thứ tự mặc định của tuple đã đúng và không cần key=. Mỗi lần sorted gọi key là một lời gọi hàm nhân với m , và đó là khoản đắt nhất có thể thêm vào một phép sắp xếp vốn chạy bằng mã C.

7. DSU offline, chạy ngược thời gian

Bài J, và đây là ý tưởng đáng giá nhất của phần DSU.

DSU **hợp được nhưng không tách được**. Đề bắt xóa dần các cạnh rồi hỏi số thành phần sau mỗi lần xóa thì bạn bí, vì không có thao tác tách.

Mẹo: nếu biết trước toàn bộ truy vấn, hãy **chạy ngược thời gian**. Bắt đầu từ đồ thị còn lại sau khi đã xóa hết, rồi thêm dần các cạnh bị xóa theo thứ tự ngược. Mỗi lần thêm là một phép hợp bình thường, và các đáp án lần lượt hiện ra theo thứ tự ngược, chỉ việc lật lại khi in.

Kỹ thuật này có tên chung là **xử lý offline**: đọc hết truy vấn rồi mới trả lời, theo một thứ tự thuận tiện hơn thứ tự đề cho. Nó xuất hiện rất nhiều ở các bài khó, và đây là ví dụ đơn giản nhất để làm quen.

Chỗ dễ nhầm nhất là **thứ tự hai dòng** trong vòng chạy ngược. Trạng thái ngay sau lần xóa thứ i là đồ thị **chưa có** cạnh $\text{del}[i]$, nên phải ghi đáp án **trước** khi thêm cạnh đó vào. Đảo hai dòng là lệch đáp án đi một bước.

8. Fenwick tree và phép lowbit

Dãy thay đổi liên tục, vừa cập nhật một phần tử vừa hỏi tổng một đoạn. Tổng tiền tố tính không dùng được, vì mỗi lần cập nhật phải dựng lại cả bảng.

Fenwick giải cả hai thao tác trong $O(\log n)$ với hai vòng `while` ngắn:

```
t = [0] * (n + 1)      # 1-based, cell 0 is never used

# add v at position i
while i <= n:
    t[i] += v
    i += i & -i

# prefix sum up to i
s = 0
while i > 0:
    s += t[i]
    i -= i & -i
```

Phép $i \& -i$ lấy ra bit 1 thấp nhất của i , gọi là lowbit. Ô $t[i]$ giữ tổng của khối gồm $\text{lowbit}(i)$ phần tử kết thúc tại i . Từ đó:

- Tính tổng tiền tố: đứng ở i , cộng khối của nó rồi nhảy về $i - \text{lowbit}(i)$, tức xóa bit thấp nhất. Mỗi bước xóa một bit nên nhiều nhất $\log n$ bước.
- Cập nhật: đứng ở i , cộng vào khối của nó rồi nhảy tới $i + \text{lowbit}(i)$, là khối cha chứa nó.

Chỉ số bắt buộc bắt đầu từ 1. Với $i = 0$ thì $\text{lowbit}(0) = 0$ và vòng cập nhật không bao giờ tiến. Ở C++ đây là lỗi treo chương trình; ở Python nó cũng treo, và bộ chấm gọi là TLE, một cái tên gây hiểu nhầm hoàn toàn vì chương trình không hề chậm, nó đứng yên.

Một chi tiết riêng của Python. Đừng dựng cây bằng n lời gọi `add`. Viết mỗi giá trị vào ô của nó rồi đẩy mỗi ô lên cha đúng một lần, tức $O(n)$ thay cho $O(n \log n)$:

```
for i in range(1, n + 1):
    t[i] += int(data[1 + i])
    j = i + (i & -i)
    if j <= n:
        t[j] += t[i]
```

Do trên test 09 của bài F, tốt nhất trong bốn lượt: **0,34 s** cách này, **0,46 s** với n lần `add`. Không phải khoảng cách sống chết, nhưng nó miễn phí và code còn ngắn hơn.

9. Ba biến thể Fenwick

Đếm nghịch thế (bài G). Quét từ trái sang phải, với mỗi phần tử mới hỏi trong số đã đi qua có bao nhiêu giá trị **lớn hơn** nó. Số đó bằng số phần tử đã đi qua trừ số phần tử có hạng nhỏ hơn hoặc bằng. Giá trị tới 10^9 nên phải nén tọa độ trước.

Ở Python, nén tọa độ là hai dòng, và cái bảng tra nên là dict chứ không phải bisect:

```
phan_biet = sorted(set(a))
hang = {v: i for i, v in enumerate(phan_biet, 1)}
```

Một lời gọi bisect_left cho mỗi phần tử là n lời gọi hàm. Đo trên test 09 của bài G, tốt nhất trong ba lượt: **0,31 s** với dict, **0,34 s** với bisect_left.

Cập nhật đoạn, truy vấn điểm (bài H). Đặt Fenwick lên **mảng hiệu** thay vì mảng gốc. Cộng v vào đoạn $[l, r]$ thành hai cập nhật điểm, ở l và ở $r + 1$; đọc $a[i]$ thành một tổng tiền tố. Đây đúng là kỹ thuật imos của buổi 2, nhưng làm được trực tuyến. Mảng phải dài $n + 1$ ô để chỗ $r + 1$ luôn hợp lệ khi r bằng n .

Tìm phần tử thứ k (bài I). Thay vì tìm kiếm nhị phân bên ngoài rồi gọi tổng tiền tố ở mỗi bước, hãy **đi xuống chính cây Fenwick**: bắt đầu từ lũy thừa hai lớn nhất, mỗi bước thử bước tới nếu khối mà ô đó phủ vẫn nằm gọn trong k.

```
vt = 0
pw = DAU # the largest power of two not above MAXV
while pw > 0:
    j = vt + pw
    if j <= MAXV and t[j] < x:
        vt = j
        x -= t[j]
    pw >>= 1
# vt is the largest index whose prefix count is still below k
```

Cách này $O(\log n)$ so với $O(\log^2 n)$. Với q bằng $2 \cdot 10^5$ thì **cả hai đều kịp trong giới hạn 2 giây của buổi này**, nên đừng học nó vì tốc độ; học vì mẫu đi xuống cây xuất hiện lại ở rất nhiều bài khó.

Dấu so sánh phải là **nhỏ hơn thật sự**. Dùng nhỏ hơn hoặc bằng thì bước quá một nấc và trả về giá trị liền sau.

10. Khi nào tổng tiền tố tĩnh là đủ

Nên hỏi câu này trước khi gõ Fenwick, vì tổng tiền tố tĩnh vừa ngắn hơn vừa nhanh hơn.

Tình huống	Dùng gì
Dãy không đổi, chỉ hỏi tổng đoạn	tổng tiền tố tĩnh, $O(1)$ mỗi truy vấn
Cập nhật hết rồi mới hỏi	mảng hiệu, $O(1)$ mỗi cập nhật
Cập nhật và truy vấn xen kẽ	Fenwick tree, $O(\log n)$ mỗi thao tác
Cần cực trị đoạn, không phải tổng	segment tree, buổi sau

Fenwick chỉ làm được với phép toán **có nghịch đảo**, vì truy vấn đoạn là hiệu hai tổng tiền tố.

Ở Python còn một lựa chọn thứ năm mà bản C++ không có: với dãy tĩnh, `itertools.accumulate` dựng bảng tổng tiền tố bằng mã C, nhanh hơn hẳn một vòng `for` tự viết. Nhưng nó chỉ dùng được khi dãy không đổi, nên nó không thay được Fenwick ở buổi này.

11. Đọc dữ liệu và ghi kết quả

Cả mười bài của buổi này đọc từ 2 tới 5 MB dữ liệu, nên phần vào ra không phải chuyện phụ.

```
data = sys.stdin.buffer.read().split()
```

Đọc hết đầu vào thành một khối byte rồi tách một lần, sau đó đi bằng con trỏ k và ép kiểu khi cần. Đo trên test 09 của bài B, $2 \cdot 10^5$ dòng, tốt nhất trong ba lượt:

Cách đọc	Thời gian
<code>sys.stdin.buffer.read().split()</code>	0,04 s
<code>sys.stdin.readline()</code> trong vòng lặp	0,06 s
<code>input()</code> trong vòng lặp	0,15 s

`input()` đắt gấp ba, và con số ấy là **toàn bộ** thời gian chạy của một bài DSU đã xong. Nói cách khác: ở buổi này, cách đọc dữ liệu tốn nhiều hơn chính thuật toán.

Ba bài A, F, H trộn nhiều loại thao tác có độ dài dòng khác nhau, nên phải rẽ nhánh theo loại rồi mới tiến con trỏ. So `data[k] == b'1'` chứ đừng `int(data[k]) == 1`: đỡ một lần ép kiểu mỗi truy vấn, và nhớ tiền tố b vì `data` là list các đối tượng bytes.

Chiều ngược lại cũng vậy: gom kết quả vào một list rồi `'\n'.join` và ghi **một** lần. Gọi `print` $2 \cdot 10^5$ lần đắt hơn chính phép tính của nhiều bài trong buổi.

12. Cạm bẫy Python, ngoài phần thuật toán

Cạm bẫy	Hậu quả
<code>find</code> đệ quy	chết ở khoảng 32.000 mức, tín hiệu 11, không truy vết, bộ chấm báo RE
<code>sys.setrecursionlimit</code> rất lớn	không cứu gì, chỉ dời chỗ chết từ <code>RecursionError</code> sang tràn ngăn xếp thật
đặt <code>find</code> thành một hàm rồi gọi trong vòng nóng	mỗi lời gọi là chi phí thật, đo được khoảng hai phần trăm giây trên 400.000 lần
<code>sorted(e, key=lambda x: x[2])</code>	thêm m lời gọi hàm; đặt trọng số vào vị trí đầu tuple là xong
<code>/</code> thay <code>//</code>	<code>/</code> luôn cho số thực, và số thực mất chính xác từ khoảng $9 \cdot 10^{15}$
ghép chuỗi kết quả bằng <code>s += ...</code>	$O(n^2)$ vì chuỗi Python bất biến; dùng list rồi <code>join</code>
<code>t = [[0] * m] * n</code>	n hàng cùng trỏ vào một list

Hai dòng đầu bảng là cái riêng của buổi này. Năm dòng còn lại đúng cho mọi buổi.

Ngược lại, một cạm bẫy lớn của bản C++ **biến mất** ở đây: tràn số. Số nghịch thế đạt $2 \cdot 10^{10}$, tổng đoạn và tổng trọng số cây khung đạt $2 \cdot 10^{14}$, và bản C++ phải khai long long ở cả ba chỗ. Số nguyên Python tự nói rộng nên không có gì để khai. Đây là một trong những chỗ hiếm hoi Python vừa ngắn hơn vừa an toàn hơn.

13. Bộ khung tự hỏi khi gặp đề mới

- Đề có nói về nhóm, thành phần, hoặc gộp hai thứ lại không? Nghĩ DSU.
- Đề có thao tác **tách** không? Nếu có, xem có xử lý offline chạy ngược được không.
- Đề có cập nhật xen kẽ truy vấn trên một dãy không? Nếu có, nghĩ Fenwick.
- Phép toán của truy vấn có nghịch đảo không? Không có thì Fenwick không dùng được.
- Giá trị lớn nhưng ít giá trị phân biệt? Nén tọa độ trước, bằng dict chứ không bằng bisect.
- `find` của bạn có chữ `def` và chữ `return` cùng lúc không? Nếu có, sửa thành vòng `while` trước khi chạy thử.

- Ước lượng số phép tính. Buổi này mỗi thao tác đi nhiều nhất $\log n$ bước, tức $2 \cdot 10^5$ thao tác cho khoảng $4 \cdot 10^6$ lượt vòng lặp, cỡ 0,3 tới 0,5 giây. Nếu ước lượng ra hơn 10^7 lượt thì phải nghĩ lại trước khi gõ.

Kế tiếp: sang **Phần II** làm bài. Làm xong hãy tự chấm bằng `chay.py` trước khi mở **Phần III**.

Phần II. Mười bài tập

Đề PDF	Ở đây	Kỹ thuật chính	File nộp
A	Bài 1	DSU cơ bản	bai01.py
B	Bài 2	DSU đếm thành phần	bai02.py
C	Bài 3	DSU kèm kích thước	bai03.py
D	Bài 4	DSU phát hiện chu trình	bai04.py
E	Bài 5	Kruskal	bai05.py
F	Bài 6	Fenwick cơ bản	bai06.py
G	Bài 7	Fenwick đếm nghịch thế	bai07.py
H	Bài 8	Fenwick trên mảng hiệu	bai08.py
I	Bài 9	Fenwick tìm phần tử thứ k	bai09.py
J	Bài 10	DSU offline chạy ngược	bai10.py

Giới hạn thời gian **2 giây** cho mọi bài, gấp đôi bản C++ vì đây là nhánh Python; xem mục I.3 và I.11 để biết con số 2 từ đâu ra. Tự chấm:

```
python3 chay.py topics/05-dsu-fenwick/solutions/bai01.py
python3 chay.py topics/05-dsu-fenwick --tat-ca
```

Mọi dòng nói về **kiểu dữ liệu** trong mười đề dưới đây, chẳng hạn *tổng đạt* $2 \cdot 10^{14}$, giữ nguyên từ bản C++ và vẫn đáng đọc, nhưng ở Python chúng **không còn là cạm bẫy**: số nguyên Python tự nở rộng nên không có gì tràn. Đối lại Python có ba cạm bẫy riêng mà bản C++ không có, và chúng nằm ở mục I.3, I.11 và I.12.

Bài A. Cùng nhóm hay không

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Có n phần tử đánh số từ 1 tới n , ban đầu mỗi phần tử là một nhóm riêng. Bạn nhận q thao tác:

- 1 u v : gộp nhóm chứa u với nhóm chứa v .
- 2 u v : hỏi u và v có đang cùng nhóm không.

Dữ liệu vào

- Dòng 1: n và q ($1 \leq n, q \leq 2 \cdot 10^5$)
- q dòng tiếp theo, mỗi dòng gồm ba số nguyên như mô tả trên ($1 \leq u, v \leq n$)

Kết quả

Với mỗi thao tác loại 2, in YES hoặc NO trên một dòng.

Ví dụ

Dữ liệu vào	Kết quả
5 5	YES
1 1 2	NO
1 3 4	YES
2 1 2	
2 1 3	
2 3 4	

Lưu ý

Bài này trông đơn giản nhưng chứa cái bẫy tinh vi nhất của cả buổi. Sau khi viết xong, hãy tự chạy tay dãy thao tác: gộp 1 với 2, rồi gộp 3 với 2, rồi hỏi 1 và 2.

Giải thích

Sau hai lệnh gộp, các nhóm là 1, 2 và 3, 4, còn 5 đứng một mình. Vì vậy 1 và 2 cùng nhóm, 1 và 3 khác nhóm, 3 và 4 cùng nhóm.

Bài B. Đếm thành phần liên thông

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho đồ thị vô hướng n đỉnh và m cạnh. Hãy đếm số thành phần liên thông.

Dữ liệu vào

- Dòng 1: n và m ($1 \leq n \leq 2 \cdot 10^5, 0 \leq m \leq 2 \cdot 10^5$)
- m dòng tiếp theo, mỗi dòng hai số u và v ($1 \leq u, v \leq n$), có thể có cạnh lặp và khuyên

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
5 3 1 2 2 3 4 5	2

Lưu ý

Bạn không cần duyệt đồ thị ở cuối. Hãy nghĩ xem số thành phần thay đổi thế nào sau mỗi phép gộp thật sự.

Giải thích

Hai thành phần là 1, 2, 3 và 4, 5.

Bài C. Nhóm lớn nhất sau mỗi lần gộp

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho n đỉnh và m cạnh, thêm lần lượt từng cạnh. Sau mỗi lần thêm, hãy in kích thước của thành phần liên thông lớn nhất hiện tại.

Dữ liệu vào

- Dòng 1: n và m ($1 \leq n, m \leq 2 \cdot 10^5$)
- m dòng tiếp theo, mỗi dòng hai số u và v

Kết quả

m dòng, mỗi dòng một số.

Ví dụ

Dữ liệu vào	Kết quả
4 3	2
1 2	2
3 4	4
2 3	

Lưu ý

Kích thước lớn nhất chỉ tăng chứ không bao giờ giảm, nên bạn không cần tính lại từ đầu. Nhưng hãy cẩn thận: mảng kích thước chỉ đúng ở một loại đỉnh nhất định.

Giải thích

Gộp 1 với 2 cho một nhóm cỡ 2. Gộp 3 với 4 cho nhóm thứ hai cũng cỡ 2, nên nhóm lớn nhất vẫn là 2. Lệnh cuối nối hai nhóm đó lại thành một nhóm cỡ 4.

Bài D. Đếm cạnh thừa

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho đồ thị vô hướng n đỉnh và m cạnh, các cạnh được thêm lần lượt. Một cạnh gọi là **thừa** nếu tại thời điểm thêm nó, hai đầu của nó đã thuộc cùng một thành phần liên thông.

Hãy đếm số cạnh thừa.

Dữ liệu vào

- Dòng 1: n và m ($1 \leq n \leq 2 \cdot 10^5, 0 \leq m \leq 2 \cdot 10^5$)
- m dòng tiếp theo, mỗi dòng hai số u và v

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
4 4 1 2 2 3 1 3 3 4	1

Giải thích: cạnh 1 3 là thừa vì lúc đó 1 và 3 đã nối nhau qua đỉnh 2.

Lưu ý

Đây cũng là cách phát hiện chu trình rẻ nhất trên đồ thị vô hướng.

Bài E. Cây khung nhỏ nhất

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho đồ thị vô hướng có trọng số gồm n đỉnh và m cạnh. Hãy tính tổng trọng số của cây khung nhỏ nhất, hoặc in -1 nếu đồ thị không liên thông.

Dữ liệu vào

- Dòng 1: n và m ($1 \leq n \leq 2 \cdot 10^5, 0 \leq m \leq 2 \cdot 10^5$)
- m dòng tiếp theo, mỗi dòng gồm u, v và w ($1 \leq w \leq 10^9$)

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
4 5 1 2 1 2 3 2 3 4 3 1 4 4 1 3 5	6

Tổng trọng số có thể đạt $2 \cdot 10^{14}$.

Lưu ý

Tổng trọng số có thể đạt $2 \cdot 10^{14}$. Ngoài ra hãy nghĩ xem làm thế nào biết đồ thị có liên thông hay không mà không phải duyệt thêm lần nữa.

Giải thích

Cây khung nhỏ nhất gồm ba cạnh trọng số 1, 2 và 3.

Bài F. Cập nhật điểm, truy vấn tổng đoạn

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên và q thao tác:

- 1 i v : cộng thêm v vào phần tử thứ i .
- 2 l r : in tổng các phần tử từ vị trí l tới r .

Dữ liệu vào

- Dòng 1: n và q ($1 \leq n, q \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($|a_i| \leq 10^9$)
- q dòng tiếp theo, mỗi dòng một thao tác ($|v| \leq 10^9$)

Kết quả

Mỗi thao tác loại 2 in một dòng.

Ví dụ

Dữ liệu vào	Kết quả
5 4	15
1 2 3 4 5	25
2 1 5	13
1 3 10	
2 1 5	
2 3 3	

Lưu ý

Tổng tiền tố tính của buổi 2 không dùng được ở đây, vì mỗi lần cập nhật sẽ phải dựng lại cả bảng. Ngoài ra tổng đoạn đạt $2 \cdot 10^{14}$.

Giải thích

Tổng cả dãy ban đầu là 15. Lệnh giữa cộng 10 vào phần tử thứ ba, nên tổng thành 25, và riêng phần tử thứ ba lúc này là $3 + 10 = 13$.

Bài G. Đếm nghịch thế

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên. Hãy đếm số cặp chỉ số (i, j) với $i < j$ và $a_i > a_j$.

Dữ liệu vào

- Dòng 1: n ($1 \leq n \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($|a_i| \leq 10^9$)

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
5 5 4 3 2 1	10

Đáp án có thể đạt gần $2 \cdot 10^{10}$.

Lưu ý

Giá trị tới 10^9 nên không đánh chỉ số trực tiếp được, hãy dùng lại kỹ thuật nén tọa độ của buổi 1. Đáp án đạt gần $2 \cdot 10^{10}$.

Giải thích

Dãy giảm hoàn toàn nên mọi cặp trong $\text{binom}{52}{10} = 10$ cặp đều là nghịch thế.

Bài H. Cập nhật đoạn, truy vấn điểm

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Ban đầu mảng n phần tử đều bằng 0. Bạn nhận q thao tác:

- 1 $l\ r\ v$: cộng thêm v vào mọi phần tử từ vị trí l tới r .
- 2 i : in giá trị hiện tại của phần tử thứ i .

Dữ liệu vào

- Dòng 1: n và q ($1 \leq n, q \leq 2 \cdot 10^5$)
- q dòng tiếp theo, mỗi dòng một thao tác ($|v| \leq 10^9$)

Kết quả

Mỗi thao tác loại 2 in một dòng.

Ví dụ

Dữ liệu vào	Kết quả
5 4	1
1 1 3 2	-1
1 2 5 -1	
2 3	
2 5	

Bài này khác bài "cộng cả đoạn" của buổi 2 ở đúng một chỗ: cập nhật và truy vấn xen kẽ nhau.

Lưu ý

Bài này khác bài *cộng cả đoạn* của buổi 2 ở đúng một chỗ: cập nhật và truy vấn xen kẽ nhau. Hãy nghĩ xem ý tưởng mảng hiệu có còn dùng được không, và nếu có thì đặt cấu trúc nào lên nó.

Giải thích

Lệnh thứ nhất cộng 2 vào các vị trí 1 tới 3, lệnh thứ hai cộng -1 vào các vị trí 2 tới 5. Vị trí 3 nhận cả hai nên bằng $2 - 1 = 1$, còn vị trí 5 chỉ nhận lệnh thứ hai nên bằng -1.

Bài I. Phần tử nhỏ thứ k

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Ban đầu bạn có một đa tập rỗng. Bạn nhận q thao tác:

- 1 x : thêm một bản của giá trị x .
- 2 x : xóa một bản của giá trị x , đề bảo đảm giá trị đó đang có mặt.
- 3 k : in giá trị nhỏ thứ k trong đa tập, đề bảo đảm đa tập đang có ít nhất k phần tử.

Dữ liệu vào

- Dòng 1: q ($1 \leq q \leq 2 \cdot 10^5$)
- q dòng tiếp theo, mỗi dòng một thao tác ($1 \leq x \leq 2 \cdot 10^5$)

Kết quả

Mỗi thao tác loại 3 in một dòng.

Ví dụ

Dữ liệu vào	Kết quả
6	5
1 5	7
1 3	
1 7	
3 2	
2 5	
3 2	

Giải thích: sau ba lần thêm, đa tập là $\{3, 5, 7\}$ nên phần tử nhỏ thứ hai là 5. Sau khi xóa 5, đa tập là $\{3, 7\}$ nên phần tử nhỏ thứ hai là 7.

Lưu ý

Có cách tìm kiếm nhị phân bên ngoài cho $O(\log^2 n)$, và có cách đi thẳng xuống cây cho $O(\log n)$. Cả hai đều kịp, nhưng cách thứ hai đáng học vì nó dùng lại được ở nhiều bài khó hơn.

Bài J. Thành phần liên thông sau mỗi lần xóa cạnh

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho đồ thị vô hướng n đỉnh và m cạnh, các cạnh đánh số từ 1 tới m . Sau đó q cạnh bị xóa lần lượt, các chỉ số bị xóa đôi một khác nhau.

Sau mỗi lần xóa, hãy in số thành phần liên thông của đồ thị còn lại.

Dữ liệu vào

- Dòng 1: n, m và q ($1 \leq n, m \leq 2 \cdot 10^5, 1 \leq q \leq m$)
- m dòng tiếp theo, mỗi dòng hai số u và v
- q dòng cuối, mỗi dòng một chỉ số cạnh bị xóa

Kết quả

q dòng, mỗi dòng một số.

Ví dụ

Dữ liệu vào	Kết quả
4 3 2 1 2 2 3 3 4 2 3	2 3

Giải thích: xóa cạnh 2 thì còn hai nhóm $\{1, 2\}$ và $\{3, 4\}$. Xóa tiếp cạnh 3 thì còn $\{1, 2\}$, $\{3\}$ và $\{4\}$.

Lưu ý

Cấu trúc của buổi này gộp được nhưng không tách được. Bạn biết trước toàn bộ danh sách cạnh bị xóa, và đó là gợi ý quan trọng nhất của bài.

Phần III. Gợi ý và lời giải

Đọc từng tầng một, dừng lại ngay khi đã nghĩ ra. Code đầy đủ nằm ở Phần IV.

Bài	Cái bẫy thật sự	Riêng với Python
A	nối hai nút mà chưa tìm gốc, cắt đứt cả cây con	find đệ quy chết ở 32.000 mức
B	giảm số thành phần mà không kiểm có gộp thật không	không có
C	đọc kích thước tại chính đỉnh thay vì tại gốc	không có
D	đếm nhầm chiều, tính cạnh nối được thay vì cạnh thừa	không có
E	quên kiểm đồ thị liên thông	đừng dùng key= khi sắp cạnh
F	Fenwick đánh chỉ số từ 0 nên vòng lặp đứng yên	dựng cây $O(n)$, đừng gọi add n lần
G	đếm số phần tử nhỏ hơn thay vì lớn hơn	tra hạng bằng dict, không bằng bisect
H	trừ tại r thay vì $r + 1$	không có
I	lệch một đơn vị khi đi xuống cây Fenwick	không có
J	xử lý xóa theo chiều xuôi, mà DSU thì không tách được	không có

Bài A. Cùng nhóm hay không

Gợi ý 1. Mỗi nhóm là một cây, gốc cây đại diện cho cả nhóm. Hai phần tử cùng nhóm khi và chỉ khi cùng gốc.

Gợi ý 2. Để cây không dài ra thành chuỗi, luôn treo cây nhỏ dưới cây lớn, và trên đường tìm gốc thì cho mỗi nút trở thẳng lên ông nó.

Gợi ý 3. Viết find bằng vòng while, không bằng đệ quy. Mục I.3 nói vì sao, kèm số đo.

Lời giải. Hai list p và sz, hai vòng while tìm gốc kiểu chia đôi đường, rồi hợp theo kích thước. Chi phí khấu hao mỗi thao tác coi như hằng số.

Cái bẫy lớn nhất của cả buổi nằm ở bài này. Phép hợp phải gán cha cho một **gốc**. Nếu bạn viết $p[b] = a$ với b chưa phải gốc thì cả cây con của b bị cắt khỏi nhóm cũ và dán sang nhóm mới.

Điều đáng sợ là **nó vẫn cho kết quả đúng trong rất nhiều trường hợp**. Nếu chỉ hợp theo kiểu chuỗi, 1 với 2 rồi 2 với 3 rồi 3 với 4, thì phần tử thứ hai luôn còn là gốc và mọi thứ chạy đúng. Lỗi chỉ lộ khi một nút **đã có cha** bị đem đi hợp, chẳng hạn hợp 1 với 2 rồi hợp 3 với 2.

Đột biến bỏ hai vòng tìm gốc ấy **vượt qua cả test mẫu lẫn test cỡ tối đa $2 \cdot 10^5$ thao tác ngẫu nhiên** khi bộ đề C++ được dựng. Phải có một test ba dòng dựng riêng mới bắt được, và test đó là tests/bai01/03.in mà nhánh này dùng chung.

Python. Bản mẫu gỡ find thẳng vào main, không để thành hàm. Mục I.3 đo được hai phần trăm giây cho việc đó trên 400.000 lần gọi. Nhưng lý do chính vẫn là ngăn xếp, không phải hai phần trăm giây ấy.

Bài B. Đếm thành phần liên thông

Gợi ý 1. Ban đầu có đúng n thành phần.

Gợi ý 2. Mỗi lần hai thành phần khác nhau gộp lại, số thành phần giảm đúng một.

Gợi ý 3. Làm sao biết một phép hợp có thật sự gộp hai thành phần khác nhau?

Lời giải. Sau hai vòng tìm gốc, điều kiện $a \neq b$ chính là câu trả lời cho gợi ý 3. Chỉ giảm comp bên trong nhánh ấy.

Nếu bạn luôn giảm mà không kiểm thì mỗi cạnh lặp và mỗi khuyên cũng bị tính, và trên dữ liệu có nhiều cạnh lặp đáp án tụt xuống âm. Một đáp án âm cho câu hỏi "có bao nhiêu thành phần" là thứ duy nhất trong buổi này tự nó kêu lên rằng nó sai.

Không cần duyệt gì ở cuối. Nhiều người viết thêm một vòng đếm số gốc, cũng đúng nhưng thừa.

Bài C. Nhóm lớn nhất sau mỗi lần gộp

Gợi ý 1. Kích thước lớn nhất chỉ tăng chứ không bao giờ giảm.

Gợi ý 2. Sau một phép hợp, chỉ đúng một nhóm đổi kích thước. Vậy chỉ cần so với nhóm đó.

Lời giải. Gộp xong thì a là gốc của nhóm vừa lớn ra, nên $sz[a]$ là con số cần so với $best$.

Phải đọc kích thước tại gốc. Mảng sz chỉ đúng tại gốc; ở các nút khác nó giữ giá trị cũ từ lúc nút đó còn là gốc. Đây là lỗi cho ra đáp án nhỏ hơn thực tế, và nó im lặng hoàn toàn. Trong bản mẫu, biến a sau hai vòng `while` đã là gốc nên chuyện này không xảy ra; nhưng nếu bạn tự gói `find` thành hàm thì đừng quên gọi nó.

Chú ý khởi tạo $best$ bằng 1 chứ không phải 0, vì với n bằng 1 và một khuyên thì nhóm lớn nhất vẫn có kích thước 1.

Bài D. Đếm cạnh thừa

Gợi ý 1. Một cạnh là thừa đúng khi hai đầu của nó đã cùng thành phần tại thời điểm thêm.

Gợi ý 2. Đó chính là lúc phép hợp không gộp được gì, tức $a == b$.

Lời giải. `if a == b`: thua $+= 1$, phần còn lại là DSU chuẩn. Đây cũng là cách phát hiện chu trình rõ nhất trên đồ thị vô hướng; nếu đề chỉ hỏi có chu trình hay không thì dừng ngay lần đầu gặp $a == b$.

Chú ý chiều của điều kiện. Đếm nhằm chiều cho ra số cạnh nối được, một con số cũng hợp lý nên không có gì báo động. Cách tự kiểm nhanh: với một đồ thị liên thông, số cạnh nối được luôn đúng bằng $n - 1$; nếu đáp án của bạn hay rơi vào đúng con số ấy thì bạn đang đếm nhằm chiều.

Bài E. Cây khung nhỏ nhất

Gợi ý 1. Sắp các cạnh theo trọng số tăng dần.

Gợi ý 2. Duyệt và lấy cạnh nào nối hai thành phần khác nhau. Cạnh nối hai đỉnh đã cùng thành phần thì bỏ, vì nó tạo chu trình.

Gợi ý 3. Làm sao biết đồ thị có liên thông không, mà không phải duyệt thêm lần nữa?

Lời giải. Kruskal, $O(m \log m)$. Đếm số cạnh đã lấy; nếu khác $n - 1$ thì đồ thị không liên thông và in -1 .

Vì sao tham lam đúng, phát biểu bằng một câu: cạnh nhẹ nhất bắc qua một lát cắt bất kỳ luôn thuộc một cây khung nhỏ nhất nào đó. Chứng minh bằng đối số đối chỗ: nếu một cây khung tối ưu không chứa cạnh đó, thêm nó vào tạo một chu trình đi qua lát cắt, và bỏ cạnh nặng hơn trên chu trình ấy cho ra cây khung không nặng hơn.

Hai chỗ mất điểm ở bản C++, và chỉ một chỗ còn sống ở bản Python:

- **Quên kiểm số cạnh đã dùng.** Đồ thị không liên thông vẫn cho ra một con số, là tổng trọng số của rừng khung, và con số đó trông hoàn toàn hợp lý. Bẫy này còn nguyên ở Python.

- **Tổng để int.** Đạt $2 \cdot 10^{14}$. Bấy này **không tồn tại** ở Python.

Python. Dựng danh sách cạnh bằng một lời gọi `sorted` trên `zip` của ba lát cắt, trọng số đặt đầu tuple để khỏi cần `key=`. Xem mục I.6. Có thể dừng vòng lặp ngay khi đã lấy đủ $n - 1$ cạnh; trên test lớn của bài này điều đó cắt bớt phần đuôi của danh sách đã sắp.

Bài F. Cập nhật điểm, truy vấn tổng đoạn

Gợi ý 1. Tổng tiền tố tính không dùng được, vì mỗi lần cập nhật phải dựng lại cả bảng.

Gợi ý 2. Cần một cấu trúc mà cả cập nhật lẫn truy vấn đều tốn $O(\log n)$.

Gợi ý 3. Fenwick tree, xem mục I.8.

Lời giải. Truy vấn đoạn bằng hiệu hai tổng tiền tố. Bản mẫu gộp luôn hai vòng tổng tiền tố vào một chỗ, cộng ở r và trừ ở $l - 1$, nên không có lời gọi hàm nào trong vòng nóng.

Ba chỗ mất điểm:

- **Chỉ số phải bắt đầu từ 1.** Với $i = 0$ thì $i \& -i$ bằng 0, vòng cập nhật không bao giờ tiến và chương trình đứng yên mãi mãi. Bộ chấm gọi đó là TLE, một cái tên gây hiểu nhầm: chương trình không chậm, nó không chạy.
- **Nhầm $i -= i \& -i$ thành $i -= 1$.** Kết quả vừa sai vừa chậm thành $O(n)$ mỗi truy vấn.
- **Kiểu dữ liệu.** Tổng đoạn đạt $2 \cdot 10^{14}$, và ở Python chuyện này không cần quan tâm.

Python. Dựng cây trong $O(n)$ thay vì n lời gọi `add`: xem mục I.8, đo được 0,34 s so với 0,46 s.

Bài G. Đếm nghịch thế

Gợi ý 1. Quét từ trái sang phải. Khi tới phần tử thứ i , bạn cần biết trong i phần tử đã đi qua có bao nhiêu giá trị **lớn hơn** nó.

Gợi ý 2. Số đó bằng i trừ đi số phần tử có giá trị nhỏ hơn hoặc bằng nó.

Gợi ý 3. Giá trị tới 10^9 nên không đánh chỉ số trực tiếp được. Nén tọa độ trước, rồi đặt Fenwick lên hạng.

Lời giải. Với mỗi i , cộng $i - \text{pref}(\text{hang}[a[i]])$ vào biến đếm rồi thêm 1 vào ô `hang[a[i]]`. Độ phức tạp $O(n \log n)$.

Hai chỗ mất điểm:

- **Đếm nhầm chiều.** Cộng thẳng `pref(hang[a[i]])` là đếm số phần tử nhỏ hơn hoặc bằng, tức đếm cặp thuận thế. Trên một dãy giảm hoàn toàn, cách sai cho ra 0 còn cách đúng cho ra $n(n-1)/2$, nên test mẫu của đề bắt được ngay.
- **Biến đếm để int.** Đạt gần $2 \cdot 10^{10}$ ở bản C++, và không tồn tại ở bản Python.

Python. Bảng hạng là `dict`, không phải `bisect_left`: 0,31 s so với 0,34 s trên test 09, xem mục I.9.

Bài H. Cập nhật đoạn, truy vấn điểm

Gợi ý 1. So bài này với bài cộng cả đoạn của buổi 2. Khác nhau ở đúng chỗ nào?

Gợi ý 2. Ở buổi 2 mọi cập nhật đến trước mọi truy vấn nên mảng hiệu là đủ. Ở đây chúng xen kẽ nhau.

Gợi ý 3. Vẫn dùng mảng hiệu, nhưng đặt Fenwick lên chính mảng hiệu đó.

Lời giải. Cộng v trên $[l, r]$ thành hai cập nhật điểm, $+v$ tại l và $-v$ tại $r + 1$. Đọc $a[i]$ thành một tổng tiền tố. Cả hai đều $O(\log n)$.

Phải khai Fenwick dài $n + 1$ để ô $r + 1$ luôn hợp lệ khi r bằng n . Đây đúng là cái bẫy $d[r + 1]$ của buổi 2, chỉ đổi vò. Ở Python nó còn có một mặt nạ nữa: nếu bạn để mảng đúng n ô thì vòng `while i <= n` đơn giản là không chạy lần nào khi i bằng $n + 1$, nên **không có `IndexError`**, không có gì báo động, chỉ có một đáp án sai ở mép phải.

Viết `-v` tại r thay vì $r + 1$ là lỗi lệch một đơn vị: đoạn cập nhật hụt mất phần tử cuối. Test mẫu vẫn có thể qua nếu không có truy vấn nào rơi đúng vào mép phải.

Bài I. Phần tử nhỏ thứ k

Gợi ý 1. Giá trị nằm trong khoảng nhỏ, nên đặt Fenwick đếm số lần xuất hiện của từng giá trị.

Gợi ý 2. Phần tử nhỏ thứ k là giá trị nhỏ nhất mà tổng tiền tố đếm tới nó đạt k .

Gợi ý 3. Có thể tìm kiếm nhị phân bên ngoài rồi gọi tổng tiền tố mỗi bước, cho $O(\log^2 n)$. Nhưng có cách đi thẳng xuống cây Fenwick cho $O(\log n)$.

Lời giải. Đi từ lũy thừa hai lớn nhất xuống, mỗi bước thử bước tới nếu khối mà ô đó phủ vẫn nằm gọn trong k . Kết thúc, vt là chỉ số lớn nhất có tổng tiền tố còn nhỏ hơn k , nên đáp án là $vt + 1$.

Dấu so sánh phải là nhỏ hơn thật sự. Dùng nhỏ hơn hoặc bằng thì bạn bước quá một nấc và trả về giá trị liền sau. Đây là lỗi lệch một đơn vị chỉ lộ ra khi k rơi đúng vào biên của một khối, nên nó sống sót qua phần lớn test viết vôi.

Một lời cảnh báo về động cơ. Với q bằng $2 \cdot 10^5$, cách $O(\log^2 n)$ cũng thừa sức lọt giới hạn 2 giây của buổi này. Đừng học cách đi xuống cây vì tốc độ, hãy học vì nó là một mẫu dùng lại được; nói nó "cần thiết để kịp giờ" ở bài này là nói quá, và chưa ai đo được điều đó.

Bài J. Thành phần liên thông sau mỗi lần xóa cạnh

Gợi ý 1. DSU gộp được nhưng không tách được. Vậy bài này không giải trực tiếp được.

Gợi ý 2. Bạn biết trước toàn bộ danh sách cạnh bị xóa. Điều đó cho phép làm gì?

Gợi ý 3. Chạy ngược thời gian: bắt đầu từ đồ thị còn lại sau khi đã xóa hết, rồi thêm dần các cạnh theo thứ tự ngược.

Lời giải. Dùng đồ thị sống sót, đếm số thành phần của nó. Rồi duyệt danh sách xóa từ cuối về đầu: ghi đáp án hiện tại vào ô i , sau đó mới thêm cạnh `del[i]` trở lại. Độ phức tạp $O((n + m + q) \alpha(n))$.

Chỗ dễ nhầm nhất là **thứ tự gán đáp án**. Trạng thái ngay sau lần xóa thứ i chính là đồ thị chưa có cạnh `del[i]`, nên phải gán đáp án **trước** khi thêm cạnh đó vào. Đảo hai dòng là đáp án lệch đi một bước, và cả q dòng cùng lệch nên nhìn output rất khó nhận ra.

Python. Đánh dấu cạnh bị xóa bằng bytearray chứ không bằng set: một lần tra bytearray là một phép chỉ số, còn một lần tra set là một phép băm. Với m bằng $2 \cdot 10^5$ thì cả hai đều kịp, nên đây là thói quen chứ không phải chênh lệch sống chết; tôi không đo riêng chỗ này.

Phần IV. Mười code mẫu đầy đủ

Mười file này là **đúng** nội dung trong solutions/ref/, tức đúng thứ bộ chấm chạy. Công tools/qa-note-code.py so từng dòng và trượt nếu hai bên lệch nhau, vì mã in ra giấy không được bộ test nào chấm nên nó là chỗ dễ lạc hậu nhất trong cả kho.

Chú thích trong mã viết bằng tiếng Anh, vì trình dựng bản in không đặt được tiếng Việt có dấu trong khối mã: XeLaTeX sẽ bỏ hẳn những ký tự ấy khỏi trang giấy và trang in vẫn trông bình thường.

Bài A. Cùng nhóm hay không

```
# Problem A: disjoint set union, merge two groups and ask whether two items share one.
# Two optimisations, both required: path compression while walking to the root, and
# union by size so the smaller tree is hung under the bigger one.
#
# The trap this problem aims at: a union must move ROOTS. Writing p[b] = a with a b
# that is not a root cuts the whole subtree of b away from its old group. It still
# answers correctly on most data, so only a test that unites a node which ALREADY has
# a parent exposes it. Test 03 of this problem is exactly three lines long and is that
# test; the random test of size 2*10^5 does not catch it.
#
# Python note. find is a LOOP here, and it is written out inside main rather than kept
# as a function. The reason is not taste. A recursive find is limited to 1000 frames by
# default, and raising the limit only moves the failure: CPython then overflows the
# real OS stack. Measured on this machine, a recursive find survives a chain of about
# 32000 nodes and dies with signal 11 and no traceback from about 33000 on, while n
# here reaches 2*10^5. The grader can only report that as RE.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); q = int(data[1])

    p = list(range(n + 1))
    sz = [1] * (n + 1)

    out = []
    k = 2
    for _ in range(q):
        loai = data[k]
        a = int(data[k + 1]); b = int(data[k + 2]); k += 3

        # Path halving: point a node at its grandparent, then step up. One pass, no
        # call, and it leaves a in the root when the loop ends.
        while p[a] != a:
            p[a] = p[p[a]]
            a = p[a]
        while p[b] != b:
            p[b] = p[p[b]]
            b = p[b]

        if loai == b'1':
            if a != b:
                # Union by size: without it a tree can degenerate into a chain.
                if sz[a] < sz[b]:
                    a, b = b, a
                p[b] = a
                sz[a] += sz[b]
            else:
                out.append("YES" if a == b else "NO")
```

```

sys.stdout.write("\n".join(out) + "\n")

main()

```

Bài B. Đếm thành phần liên thông

```

# Problem B: number of connected components of an undirected graph.
# Start from n components and subtract one every time a union REALLY merges two
# different trees. Nothing has to be traversed at the end.
#
# The trap: decrementing the counter on every edge instead of only on a real merge.
# Repeated edges and self loops then push the answer below the true value, and on a
# graph with many duplicated edges it goes negative.
#
# Python note. find is inlined as a path halving loop for the reason explained in
# bai01.py: a recursive find dies on the real OS stack, not with RecursionError.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); m = int(data[1])

    p = list(range(n + 1))
    sz = [1] * (n + 1)

    comp = n
    k = 2
    for _ in range(m):
        a = int(data[k]); b = int(data[k + 1]); k += 2

        while p[a] != a:
            p[a] = p[p[a]]
            a = p[a]
        while p[b] != b:
            p[b] = p[p[b]]
            b = p[b]

        # This is the whole problem: the counter moves only when a differs from b.
        if a != b:
            if sz[a] < sz[b]:
                a, b = b, a
            p[b] = a
            sz[a] += sz[b]
            comp -= 1

    sys.stdout.write("%d\n" % comp)

main()

```

Bài C. Nhóm lớn nhất sau mỗi lần gộp

```

# Problem C: size of the largest component after every edge is added.

```

```

# The largest size never shrinks, and only the component that was just merged can have
# changed, so one comparison per edge is enough.
#
# The trap: reading the size at the node instead of at its root. The size array is only
# correct AT A ROOT; at any other node it keeps the stale value from the time that node
# was still a root. The answer then comes out too small, silently. Here the read is
# sz[a] right after the merge, and a is a root at that point precisely because the two
# halving loops above ran first.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); m = int(data[1])

    p = list(range(n + 1))
    sz = [1] * (n + 1)

    best = 1          # with n = 1 and a self loop the largest group still has size 1
    out = []
    k = 2
    for _ in range(m):
        a = int(data[k]); b = int(data[k + 1]); k += 2

        while p[a] != a:
            p[a] = p[p[a]]
            a = p[a]
        while p[b] != b:
            p[b] = p[p[b]]
            b = p[b]

        if a != b:
            if sz[a] < sz[b]:
                a, b = b, a
            p[b] = a
            sz[a] += sz[b]
            if sz[a] > best:
                best = sz[a]
        out.append(best)

    sys.stdout.write("\n".join(map(str, out)) + "\n")

main()

```

Bài D. Đếm cạnh thừa

```

# Problem D: count the edges that close a cycle.
# An edge is redundant exactly when its two endpoints are already in the same component
# at the moment it arrives, that is when the union merges nothing. This is also the
# cheapest cycle detector there is for an undirected graph.
#
# The trap: counting the other direction. Counting the edges that DO merge gives the
# number of tree edges, which is also a plausible looking number, so nothing on screen
# tells you the answer is the wrong one.
import sys

def main():
    data = sys.stdin.buffer.read().split()

```

```

n = int(data[0]); m = int(data[1])

p = list(range(n + 1))
sz = [1] * (n + 1)

thua = 0
k = 2
for _ in range(m):
    a = int(data[k]); b = int(data[k + 1]); k += 2

    while p[a] != a:
        p[a] = p[p[a]]
        a = p[a]
    while p[b] != b:
        p[b] = p[p[b]]
        b = p[b]

    if a == b:
        thua += 1          # same component already, this edge closes a cycle
    else:
        if sz[a] < sz[b]:
            a, b = b, a
        p[b] = a
        sz[a] += sz[b]

sys.stdout.write("%d\n" % thua)

main()

```

Bài E. Cây khung nhỏ nhất

```

# Problem E: total weight of a minimum spanning tree, Kruskal.
# Sort the edges by weight, then take an edge whenever it joins two different
# components. Complexity  $O(m \log m)$ , dominated by the sort.
#
# Two traps. Forgetting the "n - 1 edges were used" check prints the weight of a
# spanning FOREST for a disconnected graph, and that number looks perfectly
# reasonable. The C++ version also has to watch a 64 bit total of up to  $2 \cdot 10^{14}$ ; in
# Python integers never overflow, so that half of the C++ warning does not apply here.
#
# Python note. The edges are sorted with ONE call to sorted() over a zip of three
# slices of the token list. Building the list with a Python level loop is the same work
# done one object at a time. sorted() compares tuples field by field, so putting the
# weight first removes the need for a key function, which would cost one call per edge.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); m = int(data[1])

    canh = data[2:2 + 3 * m]
    # Weight first so that the default tuple order is already the order we want.
    e = sorted(zip(map(int, canh[2::3]), map(int, canh[0::3]), map(int, canh[1::3])))

    p = list(range(n + 1))
    sz = [1] * (n + 1)

    tong = 0

```

```

dung = 0
for w, a, b in e:
    while p[a] != a:
        p[a] = p[p[a]]
        a = p[a]
    while p[b] != b:
        p[b] = p[p[b]]
        b = p[b]

    if a != b:
        if sz[a] < sz[b]:
            a, b = b, a
        p[b] = a
        sz[a] += sz[b]
        tong += w
        dung += 1
        if dung == n - 1:
            break # a spanning tree is complete, the rest cannot help

# Exactly n-1 accepted edges means the graph was connected. This check is free.
sys.stdout.write("%d\n" % (tong if dung == n - 1 else -1))

main()

```

Bài F. Cập nhật điểm, truy vấn tổng đoạn

```

# Problem F: Fenwick tree, point update and range sum.
# Cell t[i] holds the sum of the block of lowbit(i) elements ending at i, where lowbit
# is i & -i, the lowest set bit. A prefix sum walks down by clearing that bit, an
# update walks up by adding it, so both touch at most log n cells.
#
# The trap: 1-based indexing is not a style choice. At i = 0 the value of i & -i is 0,
# so the update loop adds nothing and never moves, and the program hangs forever on a
# line of code that looks completely ordinary.
#
# Python note. The tree is built in O(n) rather than by n separate add() walks: write
# every value into its own cell, then push each cell into its parent once. Measured on
# test 09 of this problem, best of four runs: 0.34 s this way against 0.46 s with n
# add() walks. Both fit the limit, but the O(n) build is also the shorter code.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); q = int(data[1])

    t = [0] * (n + 1)
    for i in range(1, n + 1):
        t[i] += int(data[1 + i])
        j = i + (i & -i)
        if j <= n:
            t[j] += t[i]

    out = []
    k = 2 + n
    for _ in range(q):
        loai = data[k]
        if loai == b'1':
            i = int(data[k + 1]); v = int(data[k + 2]); k += 3

```

```

        while i <= n:
            t[i] += v
            i += i & -i
    else:
        l = int(data[k + 1]) - 1; r = int(data[k + 2]); k += 3
        s = 0
        while r > 0:
            s += t[r]
            r -= r & -r
        while l > 0:
            s -= t[l]
            l -= l & -l
        # A range sum is a difference of two prefixes, which is why a Fenwick tree
        # only works for an operation that has an inverse.
        out.append(s)

    sys.stdout.write("\n".join(map(str, out)) + "\n")

main()

```

Bài G. Đếm nghịch thế

```

# Problem G: count inversions, that is pairs  $i < j$  with  $a[i] > a[j]$ .
# Sweep left to right. When the element at position  $i$  arrives, the number of already
# seen values that are strictly GREATER than it is  $i$  minus the number of seen values
# with rank at most its own. A Fenwick tree over compressed ranks gives that in  $\log n$ .
#
# Two traps. Counting the other direction, that is adding  $\text{pref}(\text{rank})$  itself, counts the
# non inversions instead, and the number still looks plausible. The C++ version also
# needs a 64 bit counter for an answer near  $2 \cdot 10^{10}$ ; Python integers grow on their own,
# so that trap disappears on this branch.
#
# Python note. The ranks come from a dict built once over  $\text{sorted}(\text{set}(a))$ . A bisect call
# per element would be one function call per element. Measured on test 09, best of
# three: 0.31 s with the dict against 0.34 s with bisect_left. Small, but free.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    a = list(map(int, data[1:1 + n]))

    # Coordinate compression: values reach  $10^9$  so they cannot index an array directly.
    phan_biet = sorted(set(a))
    hang = {v: i for i, v in enumerate(phan_biet, 1)}
    m = len(phan_biet)

    t = [0] * (m + 1)
    nghich = 0
    for i in range(n):
        j = hang[a[i]]
        s = 0
        x = j
        while x > 0:
            s += t[x]
            x -= x & -x
        nghich += i - s
        # seen so far, minus those with rank <= this one
        while j <= m:

```

```

        t[j] += 1
        j += j & -j

    sys.stdout.write("%d\n" % nghich)

main()

```

Bài H. Cập nhật đoạn, truy vấn điểm

```

# Problem H: range update, point query.
# Put the Fenwick tree over the DIFFERENCE array instead of the array itself. Adding v
# on [l, r] becomes two point updates, at l and at r + 1, and reading a[i] becomes one
# prefix sum. This is the imos trick of session 2 made online: there the updates all
# came before the queries, here they are interleaved.
#
# Two traps. Subtracting at r instead of r + 1 loses the last element of the range, and
# a sample test misses it unless some query lands exactly on the right edge. And the
# tree must be n + 1 cells long, otherwise the update at r + 1 falls off the end when
# r equals n. That is the same d[r + 1] trap of session 2 in a new coat.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); q = int(data[1])

    # One extra cell so that index r + 1 is always legal, even for r = n.
    kt = n + 1
    t = [0] * (kt + 1)

    out = []
    k = 2
    for _ in range(q):
        loai = data[k]
        if loai == b'l':
            l = int(data[k + 1]); r = int(data[k + 2]); v = int(data[k + 3]); k += 4
            i = l
            while i <= kt:
                t[i] += v
                i += i & -i
            i = r + 1
            while i <= kt:
                t[i] -= v
                i += i & -i
        else:
            i = int(data[k + 1]); k += 2
            s = 0
            while i > 0:
                s += t[i]
                i -= i & -i
            out.append(s)

    sys.stdout.write("\n".join(map(str, out)) + "\n")

main()

```

Bài I. Phần tử nhỏ thứ k

```

# Problem I: a multiset over a small value range, with a k-th smallest query.
# Keep the count of each value in a Fenwick tree indexed by value. The k-th smallest is
# then found by DESCENDING the tree itself: start at the highest power of two and take
# a step whenever the block that cell covers still fits inside k. That is  $O(\log n)$ ,
# against  $O(\log^2 n)$  for a binary search that calls pref() at every step.
#
# The trap: the comparison in the descent must be strictly less than. Using <= steps
# one notch too far and returns the value just after the right one, and that shows up
# only when k lands exactly on a block boundary.
#
# Python note. LOG is computed from the fixed value range, and the descent is an
# unrolled walk over powers of two with no function call inside, for the usual reason:
# in a loop that runs  $2 \cdot 10^5$  times, a call costs more than the work it wraps.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    q = int(data[0])

    MAXV = 200000          # the statement guarantees  $1 \leq x \leq 2 \cdot 10^5$ 
    t = [0] * (MAXV + 1)
    LOG = 1
    while (1 << (LOG + 1)) <= MAXV:
        LOG += 1
    DAU = 1 << LOG

    out = []
    k = 1
    for _ in range(q):
        loai = data[k]
        x = int(data[k + 1]); k += 2
        if loai == b'1':
            i = x
            while i <= MAXV:
                t[i] += 1
                i += i & -i
        elif loai == b'2':
            i = x
            while i <= MAXV:
                t[i] -= 1
                i += i & -i
        else:
            vt = 0
            pw = DAU
            while pw > 0:
                j = vt + pw
                if j <= MAXV and t[j] < x:
                    vt = j
                    x -= t[j]
                pw >>= 1
            # vt is the largest index whose prefix count is still below k.
            out.append(vt + 1)

    sys.stdout.write("\n".join(map(str, out)) + "\n")

main()

```

Bài J. Thành phần liên thông sau mỗi lần xóa cạnh

```
# Problem J: components after each edge deletion, answered offline in reverse.
# A disjoint set union can merge but cannot split, so deletions cannot be processed in
# the order they are given. Run time BACKWARDS instead: build the graph that survives
# every deletion, then put the deleted edges back one by one in reverse order. Each
# re-insertion is an ordinary union, and the answers come out reversed.
# Complexity  $O((n + m + q) \alpha)$ .
#
# The trap: the order of the two lines in the backward loop. The state right after
# deletion  $i$  is the graph WITHOUT edge  $del[i]$ , so the answer has to be recorded BEFORE
# that edge is put back. Swapping the two lines shifts every answer by one step.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); m = int(data[1]); q = int(data[2])

    canh = data[3:3 + 2 * m]
    eu = list(map(int, canh[0::2]))
    ev = list(map(int, canh[1::2]))

    xoa = list(map(int, data[3 + 2 * m:3 + 2 * m + q]))
    bi_xoa = bytearray(m + 1)
    for i in xoa:
        bi_xoa[i] = 1

    p = list(range(n + 1))
    sz = [1] * (n + 1)
    comp = n

    # The graph that survives every deletion.
    for i in range(m):
        if bi_xoa[i + 1]:
            continue
        a = eu[i]; b = ev[i]
        while p[a] != a:
            p[a] = p[p[a]]
            a = p[a]
        while p[b] != b:
            p[b] = p[p[b]]
            b = p[b]
        if a != b:
            if sz[a] < sz[b]:
                a, b = b, a
            p[b] = a
            sz[a] += sz[b]
            comp -= 1

    # Walk the deletions backwards, adding each edge back.
    dap = [0] * q
    for i in range(q - 1, -1, -1):
        dap[i] = comp # the state right after deletion i
        j = xoa[i] - 1
        a = eu[j]; b = ev[j]
        while p[a] != a:
            p[a] = p[p[a]]
            a = p[a]
        while p[b] != b:
            p[b] = p[p[b]]
```

```
        b = p[b]
    if a != b:
        if sz[a] < sz[b]:
            a, b = b, a
        p[b] = a
        sz[a] += sz[b]
        comp -= 1

    sys.stdout.write("\n".join(map(str, dap)) + "\n")

main()
```